



ESCUELA POLITÉCNICA NACIONAL

TEMA:

Proyecto Final

MATERIA:

Verificación y Validación del software

INTEGRANTES:

Javier Andres Angulo Albornoz

Priscila Salome Pazmiño Santacruz

Cladio Omar Peñaherrera Llulluna

Saul Gonzalo Tualombo Benavides

Carol Abigail Velasquez Cando

FECHA

16/07/2026



Contenido

Contenido.....	¡Error! Marcador no definido.
1. Introducción.....	3
2. Objetivos.....	3
2.1 Objetivo general	3
2.2 Objetivos específicos	3
3. Herramientas utilizadas.....	3
4. Metodología de trabajo	3
5. Análisis del backend con SonarLint.....	4
5.1 Correcciones aplicadas en las clases de prueba.....	4
5.1.1 CitaControllerIntegrationTest.....	4
5.1.2 HistoriaClinicaControllerIntegrationTest.....	5
5.1.3 PacienteControllerIntegrationTest	6
5.1.4 DoctorServiceTest.....	7
5.1.5 PacienteServiceTest.....	8
5.1.6 FrontendE2ETest	9
5.1.7 Clases de prueba sin incidencias.....	10
5.2 Hallazgos en src/main que no fueron modificados	11
5.2.1 Controladores: configuración CORS permisiva	11
5.2.2 DTO: constructores vacíos	11
5.2.3 GlobalExceptionHandler.....	13
5.2.4 Modelo HistoriaClinica.....	13
5.2.5 Servicios.....	14
DoctorService	14
PacienteService	15
5.2.6 Clases de src/main sin incidencias	15
6. Análisis del frontend con ESLint.....	16
6.1 Resultado inicial	16
6.2 Configuración aplicada.....	18
6.3 Fragmentos principales de eslint.config.mjs.....	18
6.4 Resultado del frontend.....	19
7. Resumen general de resultados.....	19
8. Conclusiones.....	19
Anexo A. Reglas principales identificadas	19



1. Introducción

La revisión de calidad de un proyecto permite identificar problemas que no siempre producen un error inmediato, pero que pueden afectar la legibilidad, la mantenibilidad, la seguridad y la confiabilidad del sistema. En este trabajo se analizó el proyecto Hospital Management mediante SonarLint para el código Java del backend y ESLint para el código JavaScript del frontend.

El análisis del backend se dividió en dos partes. En las clases de prueba se aplicaron las correcciones sugeridas y se verificó el resultado con una nueva ejecución de SonarLint. En cambio, las clases ubicadas en src/main no fueron modificadas, porque correspondían al código base proporcionado por el ingeniero; en esos casos únicamente se documentaron los hallazgos, su regla, su severidad y una posible recomendación.

Para el frontend se ejecutó ESLint desde la terminal. El primer análisis presentó una gran cantidad de errores relacionados principalmente con variables globales compartidas entre archivos y con el entorno de pruebas Jest. Posteriormente se creó una configuración acorde con la estructura real del proyecto y se volvió a ejecutar el análisis.

2. Objetivos

2.1 Objetivo general

Evaluar y mejorar la calidad del código del proyecto Hospital Management mediante el uso de SonarLint y ESLint, dejando evidencia de los problemas encontrados, las correcciones aplicadas y los hallazgos que no podían modificarse.

2.2 Objetivos específicos

- Identificar problemas de mantenibilidad, confiabilidad y seguridad en el backend Java.
- Corregir los hallazgos presentes en las clases de prueba sin alterar su propósito.
- Documentar los problemas encontrados en src/main sin modificar el código proporcionado por el docente.
- Configurar ESLint de acuerdo con los archivos del navegador, los módulos compartidos y las pruebas de Jest.
- Comprobar la disminución de incidencias mediante capturas del antes y del después.

3. Herramientas utilizadas

- **SonarLint / SonarQube for IDE:** extensión utilizada en IntelliJ IDEA para analizar el código Java de manera automática y mostrar la regla, la ubicación, la categoría y el nivel visual de cada incidencia.
- **ESLint:** herramienta de análisis estático empleada para revisar el código JavaScript del frontend y las pruebas desarrolladas con Jest.
- **IntelliJ IDEA:** entorno de desarrollo utilizado para revisar y corregir el backend.
- **Terminal de Visual Studio Code:** medio utilizado para ejecutar el comando npm run lint en el frontend.

4. Metodología de trabajo

1. Se abrió cada clase del proyecto y se esperó a que el análisis automático terminara.



2. Se registró la cantidad de incidencias, la regla asociada, la clase afectada y el indicador de severidad.
3. En las clases de prueba se corrigió una incidencia a la vez y se tomó una captura antes y después del cambio.
4. Se confirmó el resultado final mediante el mensaje “No Issues”.
5. En src/main se conservaron los archivos originales y se documentaron únicamente los hallazgos.
6. En el frontend se ejecutó npm run lint, se revisaron los errores y se creó el archivo eslint.config.mjs.
7. Finalmente se repitió el análisis para comprobar la solución.

5. Análisis del backend con SonarLint

5.1 Correcciones aplicadas en las clases de prueba

En esta parte sí se realizaron cambios porque las clases correspondían a las pruebas del proyecto. Las correcciones se aplicaron de forma gradual y se verificaron con el analizador después de cada modificación.

Clase	Incidencias iniciales	Reglas principales	Resultado
CitaControllerIntegrationTest	1	java:S8694	Corregida
DoctorControllerIntegrationTest	0	No aplica	Sin incidencias
HistoriaClinicaControllerIntegrationTest	6	S1481, S1854, S2925, S8694	Corregida
PacienteControllerIntegrationTest	6	java:S8694	Corregida
CitaServiceTest	0	No aplica	Sin incidencias
DoctorServiceTest	1	java:S1128	Corregida
HistoriaClinicaServiceTest	0	No aplica	Sin incidencias
PacienteServiceTest	13	S1128 y S8694	Corregida
FrontendE2ETest	3	S125, S5786 y S116	Corregida

5.1.1 CitaControllerIntegrationTest

Problema: se utilizó el número 4 para representar el mes de abril en `LocalDate.of()`.

Regla y severidad: java:S8694, mantenibilidad, severidad media (indicador amarillo).

Corrección: se reemplazó el literal numérico por la constante `Month.APRIL`, lo que hace que la fecha sea más fácil de leer.

Antes: `LocalDate.of(1993, 4, 12)`

Después: `LocalDate.of(1993, Month.APRIL, 12)`



Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”

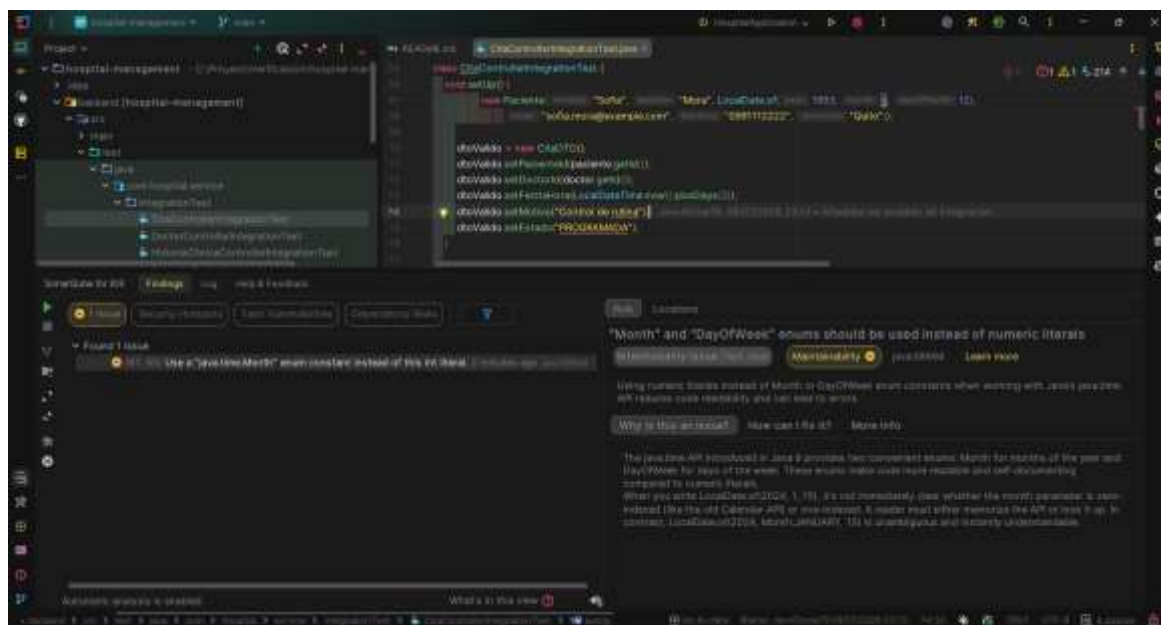


Figura 1. Antes: SonarLint detectó el uso de un mes numérico.

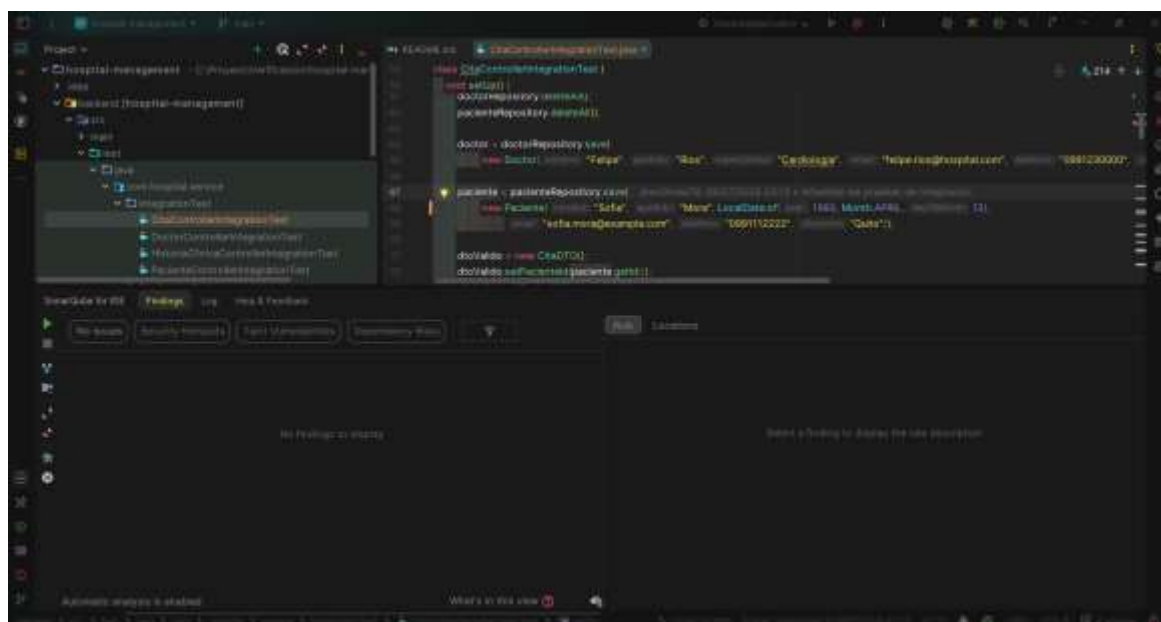


Figura 2. Después: la constante Month.APRIL fue aplicada y la clase quedó sin incidencias.

5.1.2 HistoriaClinicaControllerIntegrationTest

Problemas: se encontraron seis incidencias: variables locales sin uso, asignaciones innecesarias, uso de Thread.sleep() y un mes expresado mediante un número.

Reglas y severidad: java:S1481 y java:S8694 con severidad media; java:S1854 y java:S2925 con severidad alta, según los indicadores visuales mostrados por SonarLint.

Correcciones: se eliminó el almacenamiento en variables que no eran utilizadas, se retiraron asignaciones innecesarias, se sustituyó Thread.sleep() por una preparación determinística de los datos y se reemplazó el mes 6 por Month.JUNE.

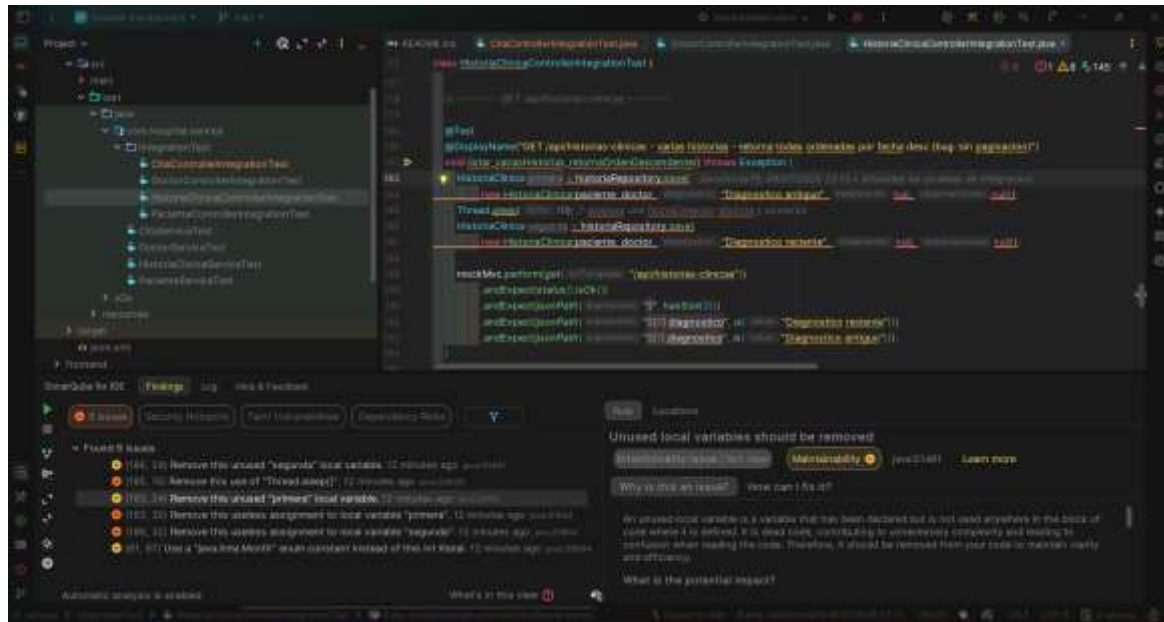


Figura 3. Antes: seis incidencias detectadas en la prueba de historias clínicas.

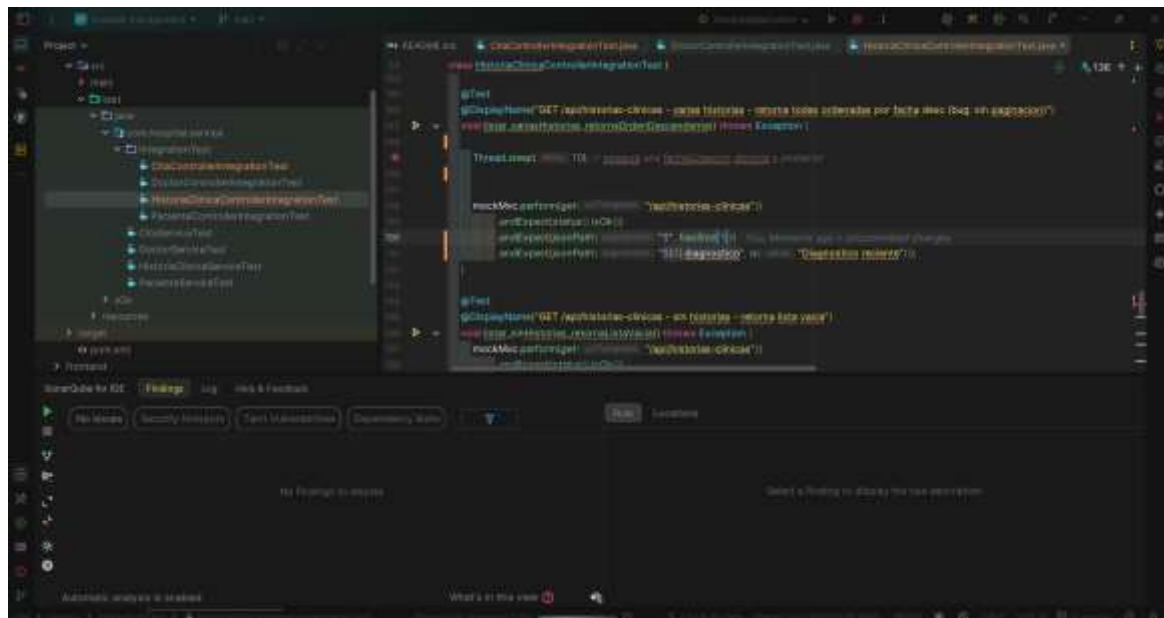


Figura 4. Después: SonarLint mostró “No Issues” en la clase analizada.

5.1.3 PacienteControllerIntegrationTest

Problema: se encontraron seis fechas en las que los meses se encontraban escritos como números enteros.

Regla y severidad: java:S8694, mantenibilidad, severidad media (indicador amarillo).

Corrección: los valores numéricos fueron reemplazados por Month.MAY, Month.JANUARY, Month.MARCH, Month.JULY y Month.FEBRUARY, según cada caso.



Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”

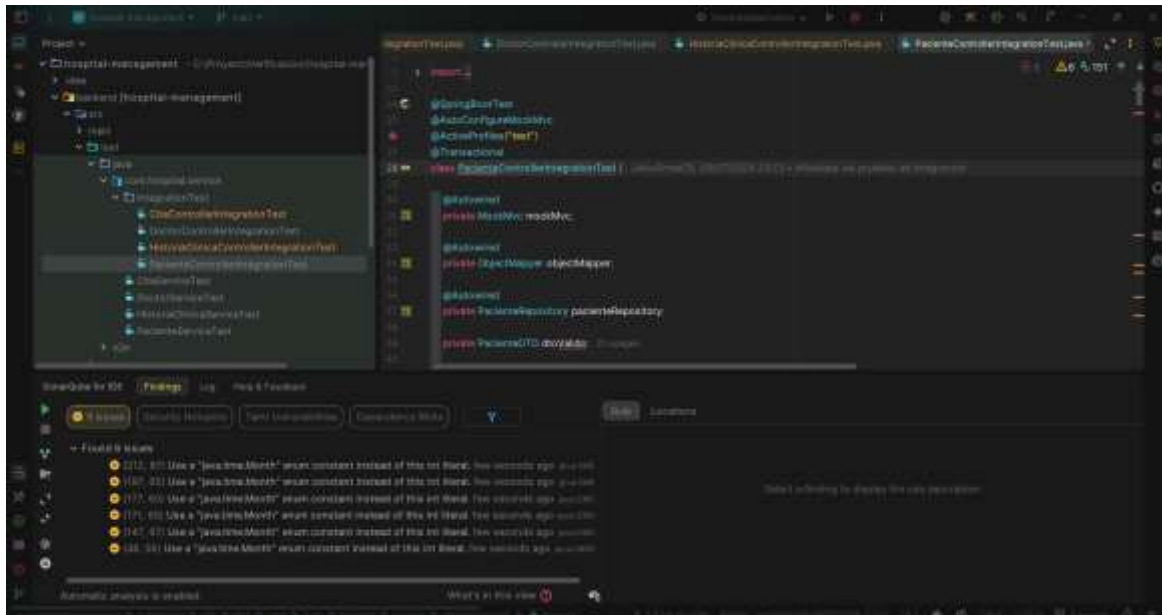


Figura 5. Antes: seis incidencias de la regla java:S8694.

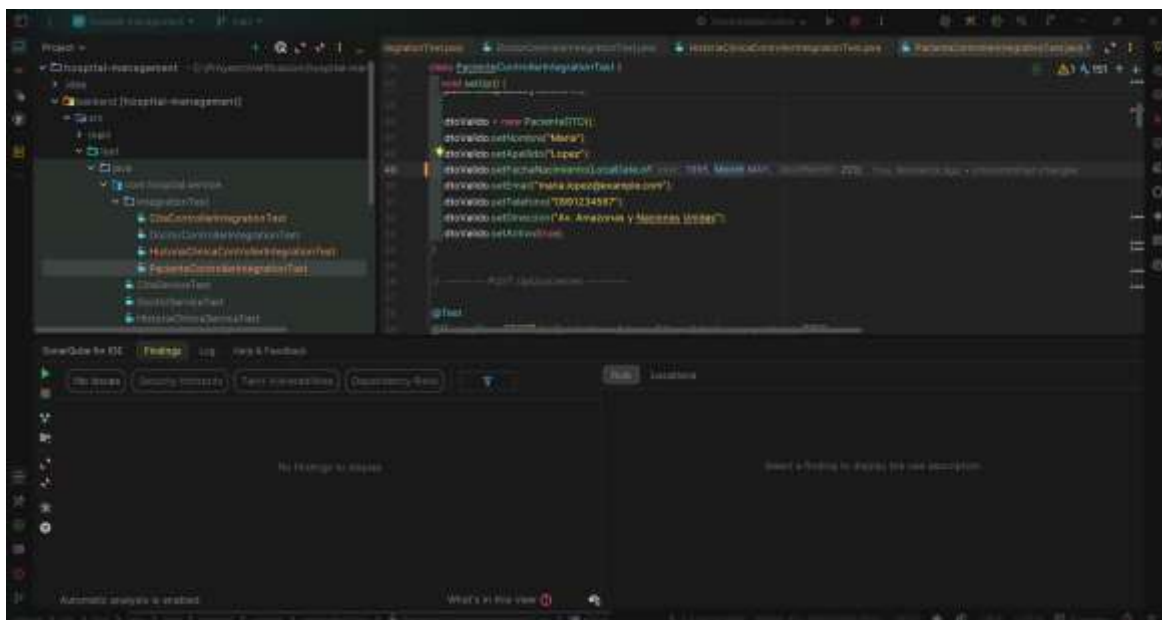


Figura 6. Después: la clase PacienteControllerIntegrationTest quedó sin incidencias.

5.1.4 DoctorServiceTest

Problema: la clase contenía la importación `org.mockito.ArgumentCaptor`, pero no era utilizada.

Regla y severidad: `java:S1128`, mantenibilidad, severidad media (indicador amarillo).

Corrección: se eliminó únicamente la línea de importación innecesaria, sin modificar los métodos de prueba.

Antes: `import org.mockito.ArgumentCaptor;`
Después: se eliminó la importación.



Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”

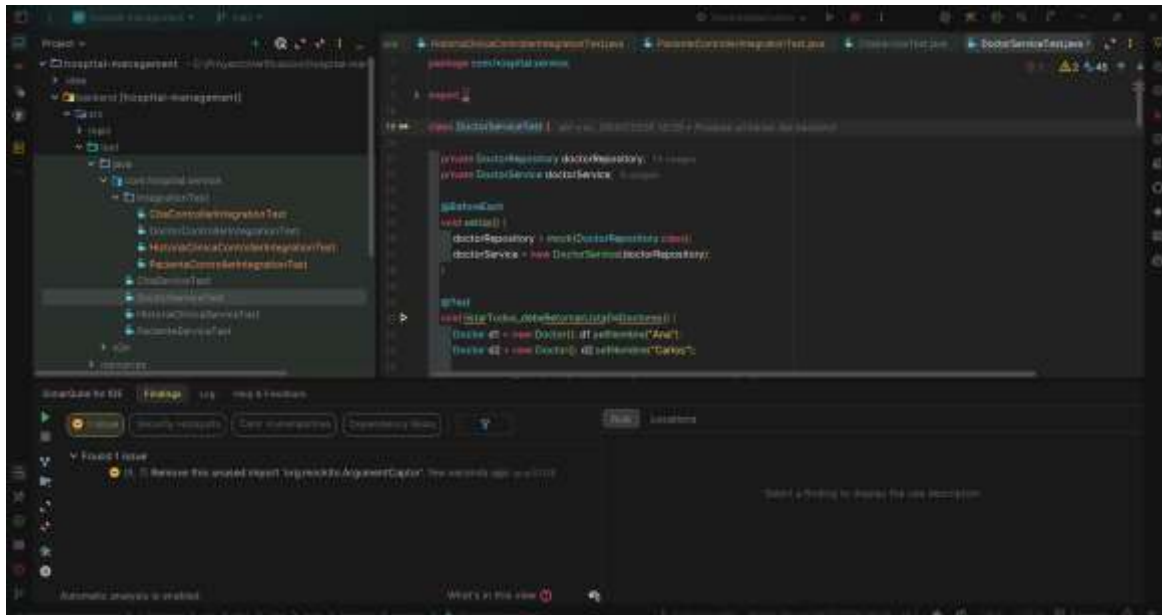


Figura 7. Antes: importación no utilizada detectada por SonarLint.

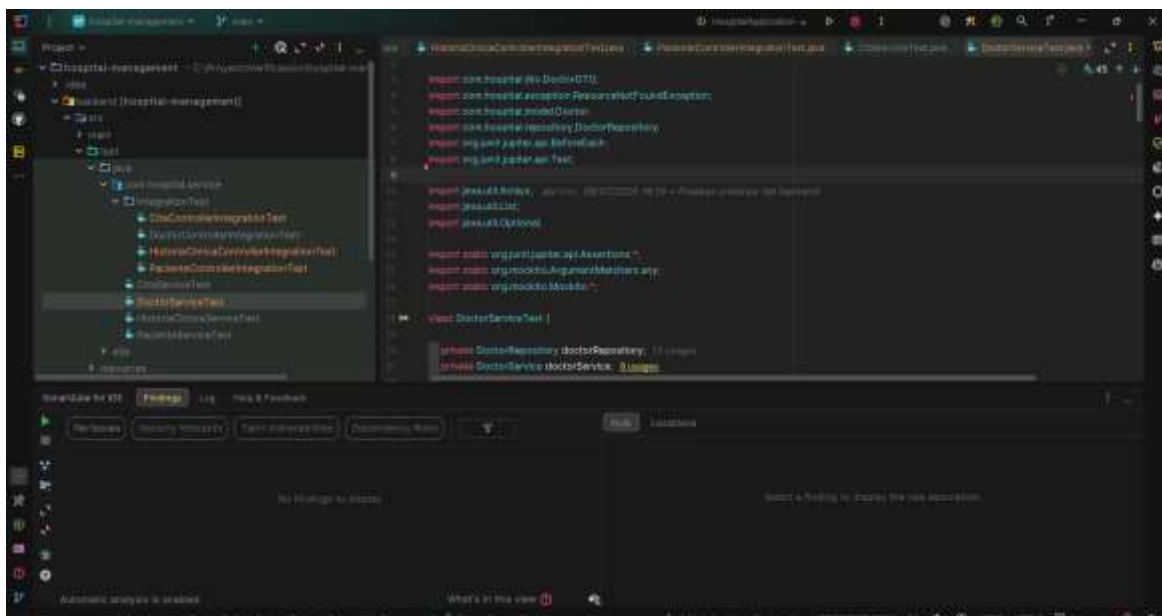


Figura 8. Después: DoctorServiceTest sin incidencias.

5.1.5 PacienteServiceTest

Problemas: se encontraron trece incidencias: una importación estática sin uso y doce meses representados mediante números.

Reglas y severidad: java:S1128 y java:S8694, categoría mantenibilidad, severidad media.

Correcciones: se eliminó la importación `assertInstanceOf` y se reemplazaron los meses numéricos por constantes de `java.time.Month`. La cantidad de incidencias disminuyó progresivamente de 13 a 0.

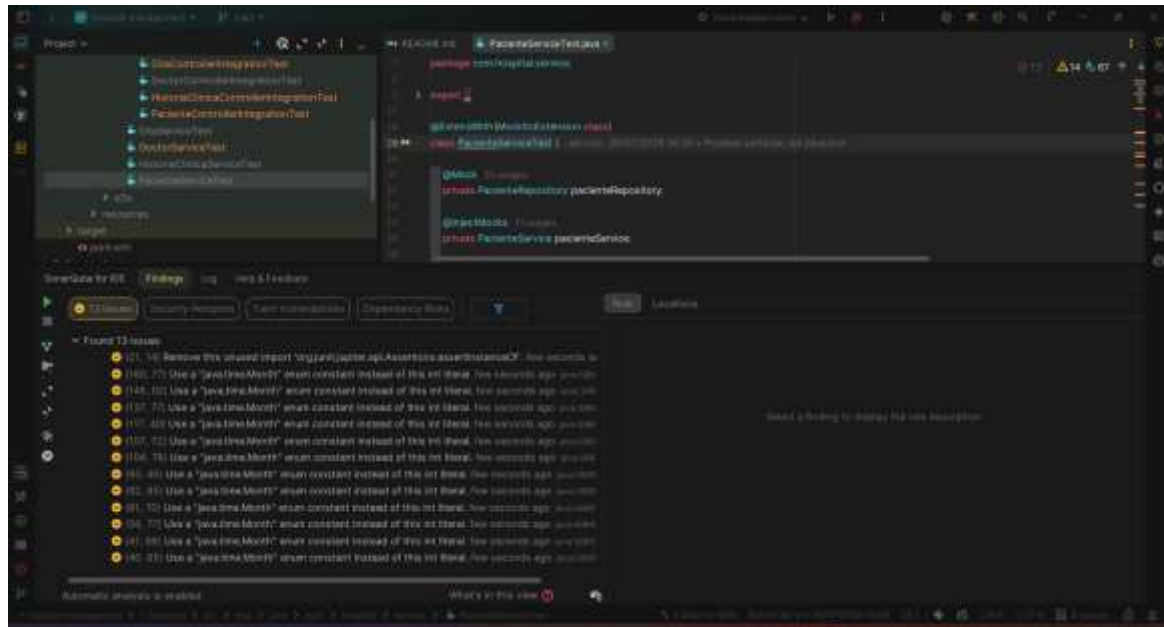


Figura 9. Antes: PacienteServiceTest presentaba trece incidencias.

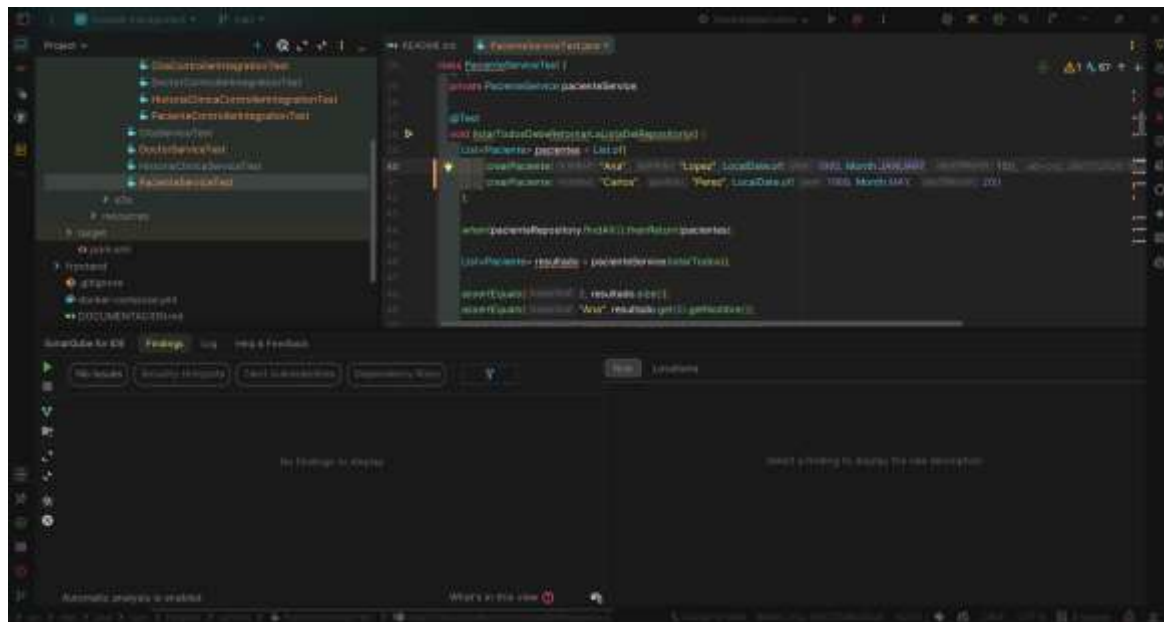


Figura 10. Después: PacienteServiceTest quedó sin incidencias.

5.1.6 FrontendE2ETest

Problemas: se encontró código comentado, un modificador public innecesario en una clase JUnit 5 y un campo que no cumplía con la convención de nombres.

Reglas y severidad: java:S125 con severidad alta, java:S5786 con severidad baja y java:S116 con severidad media.

Correcciones: se eliminó el código comentado, se retiró el modificador public y se ajustó la declaración del campo FRONTEND_URL para que cumpliera la convención de una constante.



Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”

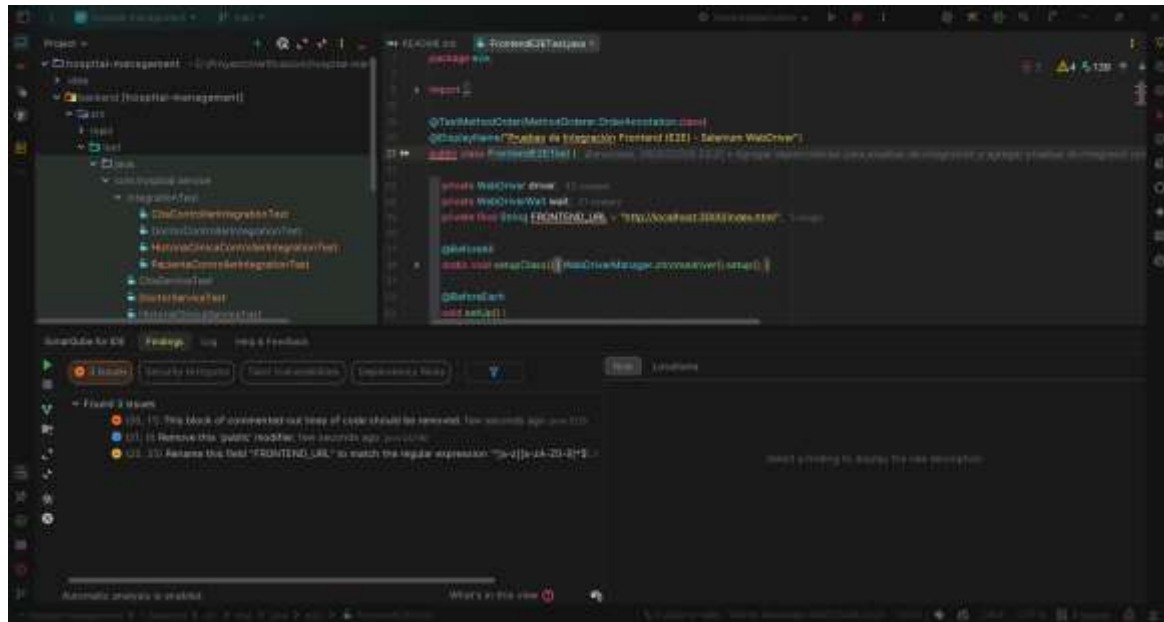


Figura 11. Antes: tres incidencias detectadas en FrontendE2ETest.

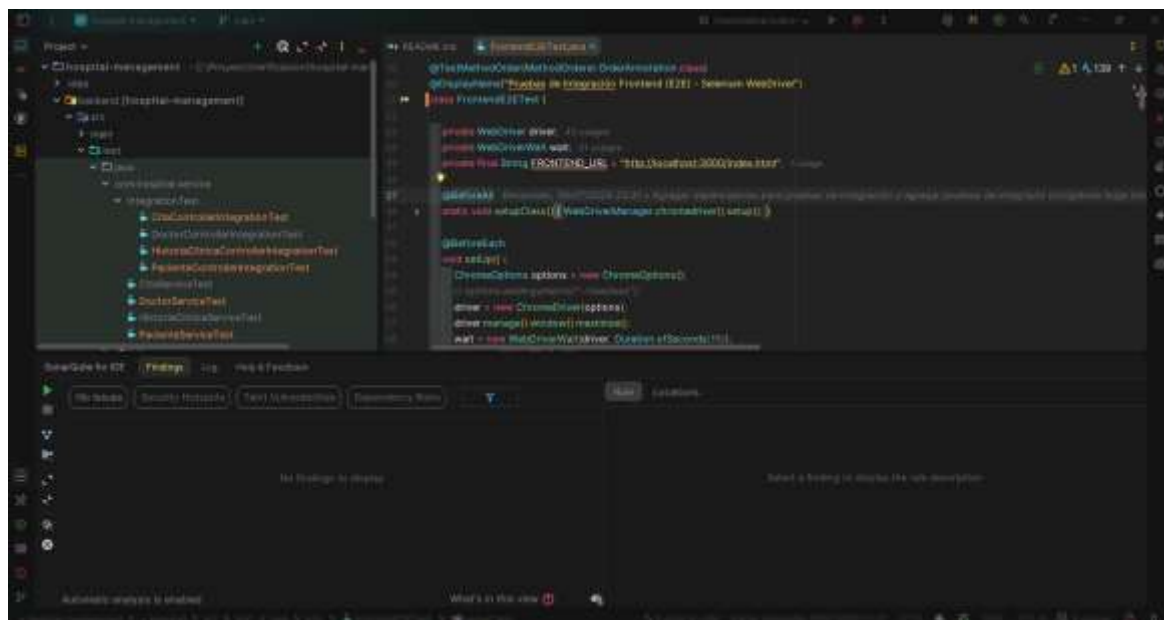


Figura 12. Después: FrontendE2ETest sin incidencias.

5.1.7 Clases de prueba sin incidencias

Las clases DoctorControllerIntegrationTest, CitaServiceTest e HistoriaClinicaServiceTest fueron revisadas y SonarLint no presentó hallazgos. Por esta razón no fue necesario modificar su código.



Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”

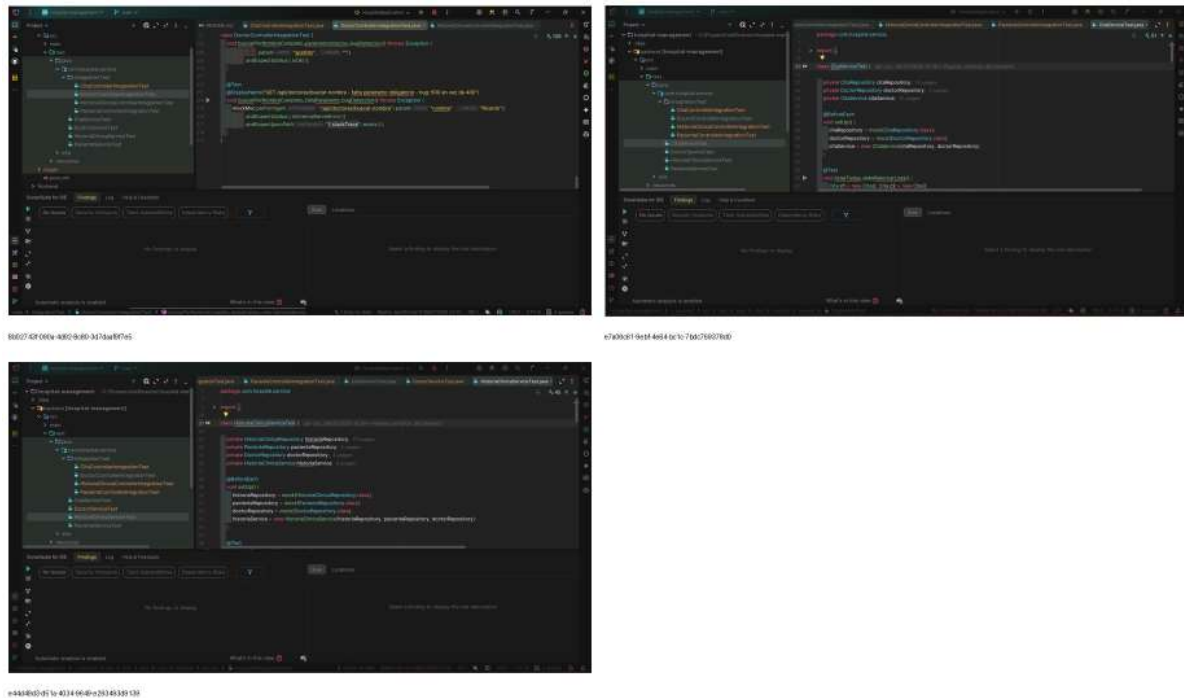


Figura 13. Evidencias de clases de prueba revisadas sin incidencias.

5.2 Hallazgos en src/main que no fueron modificados

Las clases de esta sección correspondían al código base entregado por el ingeniero. Para conservar la versión original, no se aplicaron correcciones. Los problemas se registran como observaciones y se incluye una recomendación que podría considerarse en una versión futura del proyecto.

5.2.1 Controladores: configuración CORS permisiva

5.2.2 DTO: constructores vacíos

En CitaDTO, DoctorDTO, HistoriaClinicaDTO y PacienteDTO se detectaron constructores sin instrucciones ni comentario interno.

Regla y severidad: java:S1186, mantenibilidad, severidad crítica (indicador rojo).

SonarLint recomienda justificar un método o constructor vacío. En este proyecto, una explicación adecuada sería indicar que el constructor es necesario para la deserialización realizada por Jackson u otro framework. No se aplicó el comentario porque estas clases no podían modificarse.



Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”

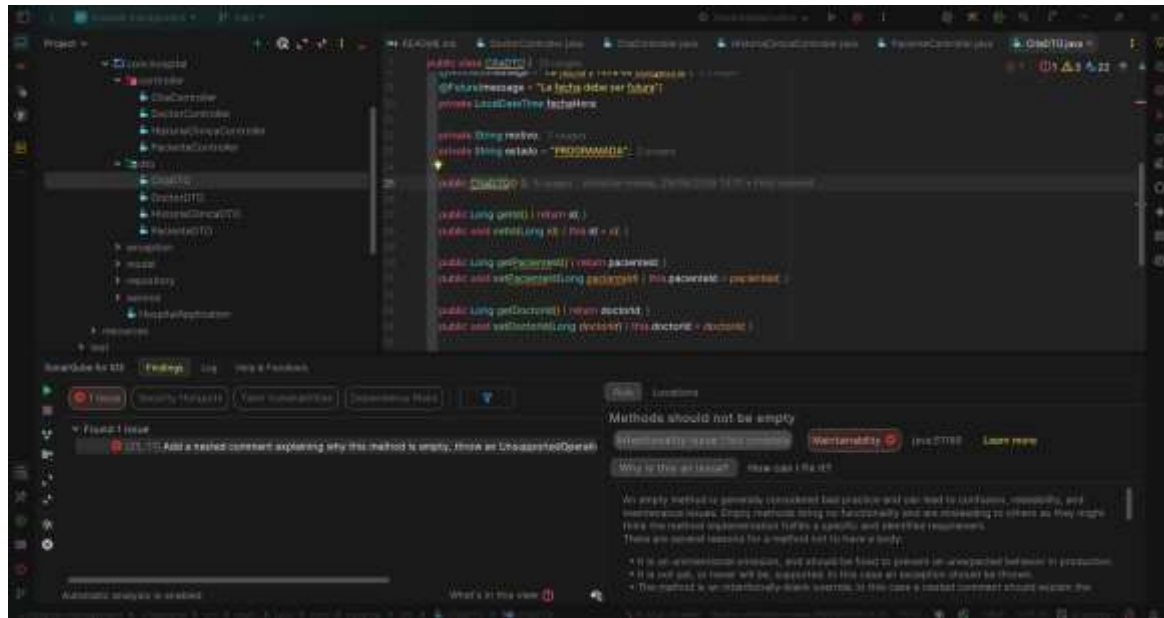


Figura 18. Constructor vacío identificado en CitaDTO.

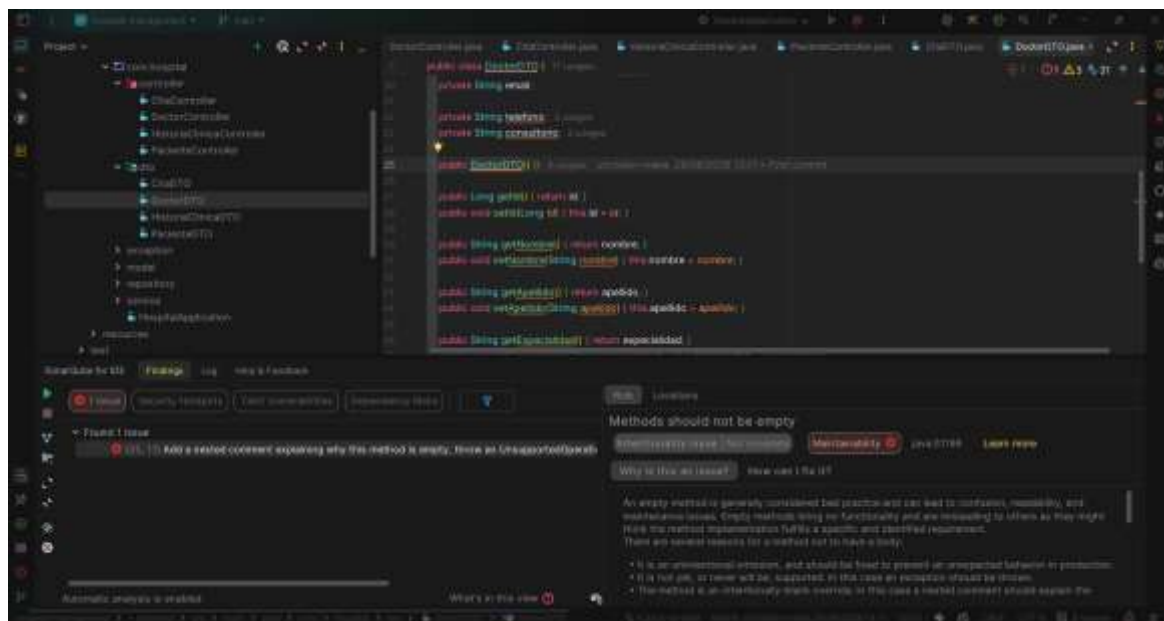


Figura 19. Constructor vacío identificado en DoctorDTO.



Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”

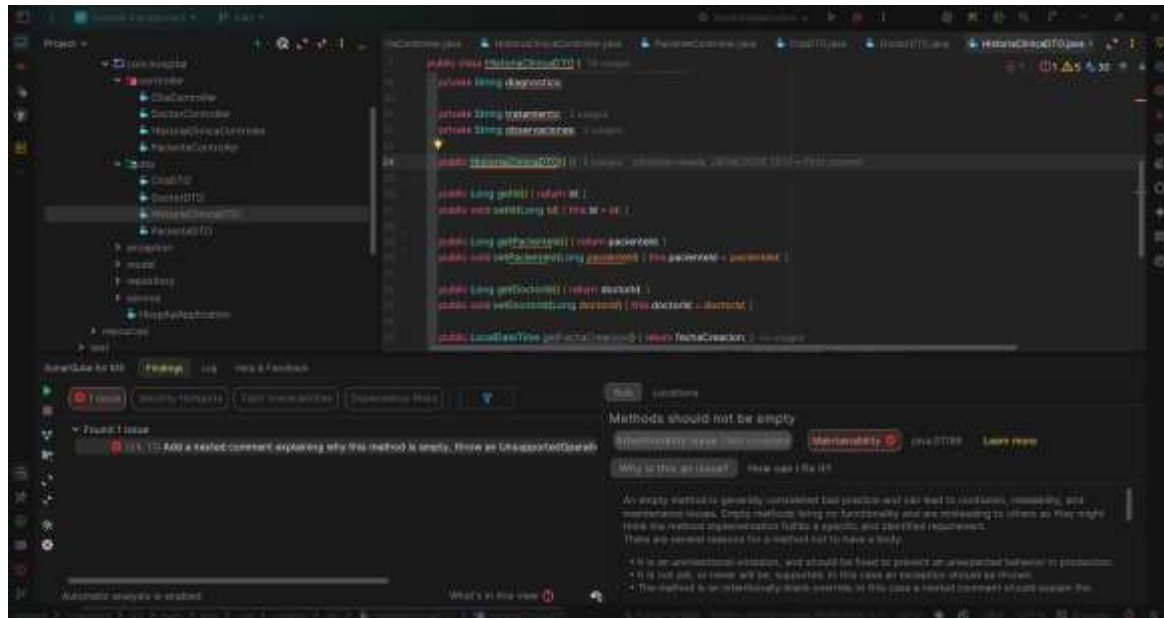


Figura 20. Constructor vacío identificado en HistoriaClinicaDTO.

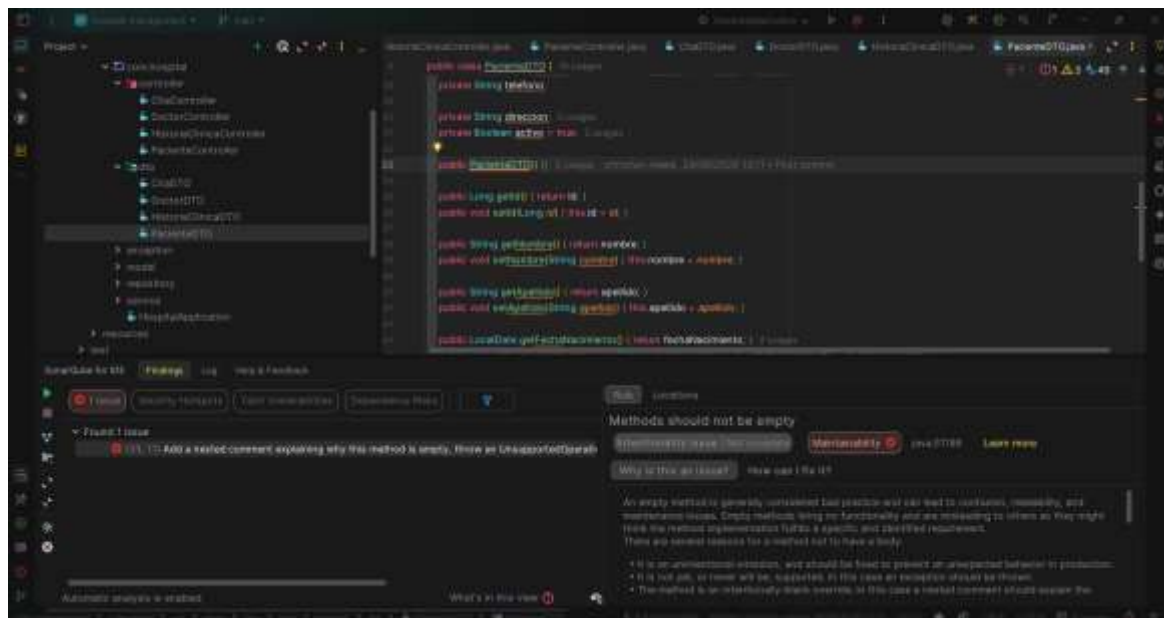


Figura 21. Constructor vacío identificado en PacienteDTO.

5.2.3 GlobalExceptionHandler

5.2.4 Modelo HistoriaClinica

Cantidad de incidencias: 2.

Regla y severidad: java:S8688, confiabilidad y mantenibilidad, severidad baja (indicador azul).

La clase inicializa fechas mediante LocalDateTime.now() sin proporcionar una zona horaria o un Clock. La recomendación sería inyectar o definir explícitamente la fuente de tiempo, especialmente para lograr pruebas repetibles.



Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”

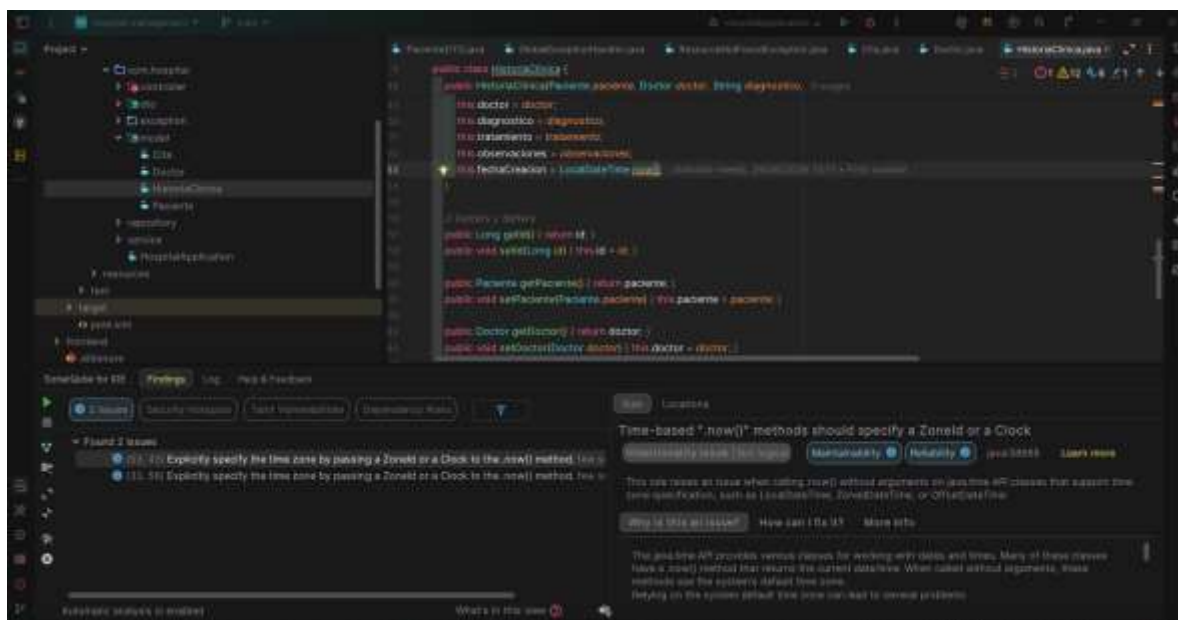


Figura 23. Hallazgos relacionados con `LocalDateTime.now()` en `HistoriaClinica`.

5.2.5 Servicios

DoctorService

Hallazgo: la consulta SQL se construye dinámicamente a partir de una cadena.

Regla y severidad: java:S2077, seguridad y mantenibilidad, severidad alta (indicador naranja).

SonarLint recomienda evitar el formato dinámico de una consulta SQL y utilizar parámetros preparados. La observación es importante porque una construcción incorrecta puede facilitar errores o inyección SQL.

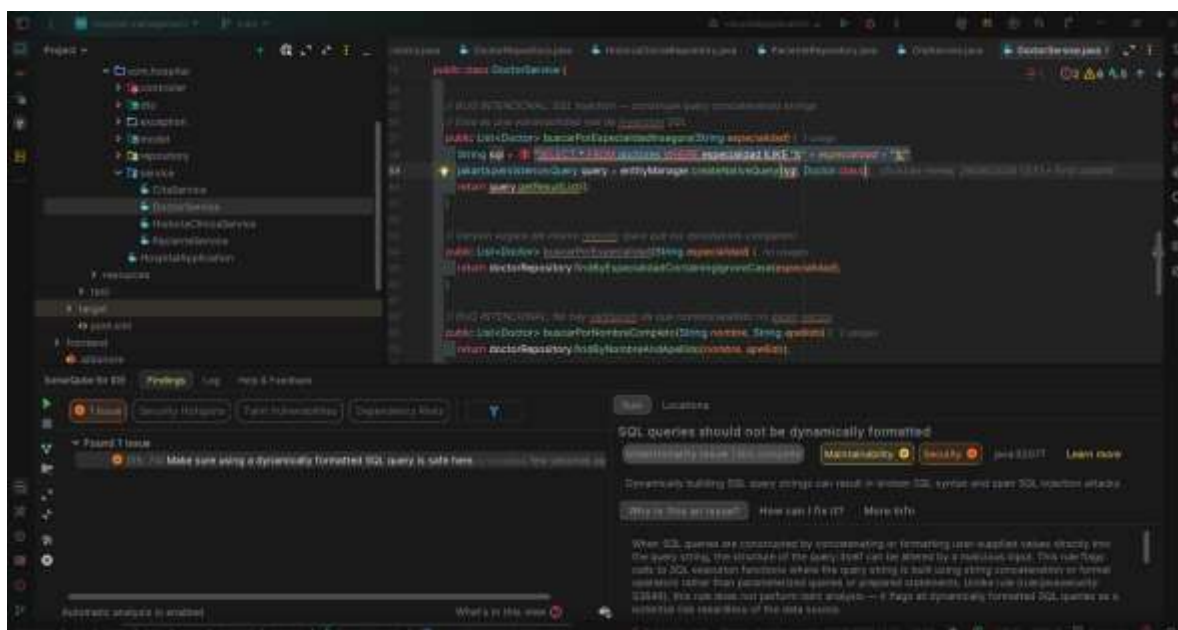


Figura 24. Consulta SQL dinámica detectada en `DoctorService`.



Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”

PacienteService

Cantidad de incidencias: 2.

Reglas y severidad: java:S1125 por un literal booleano redundante, con severidad media; y java:S8688 por LocalDate.now(), con severidad baja.

La recomendación sería simplificar la expresión booleana y utilizar una fuente de tiempo explícita. Ninguno de los cambios fue aplicado en el archivo original.

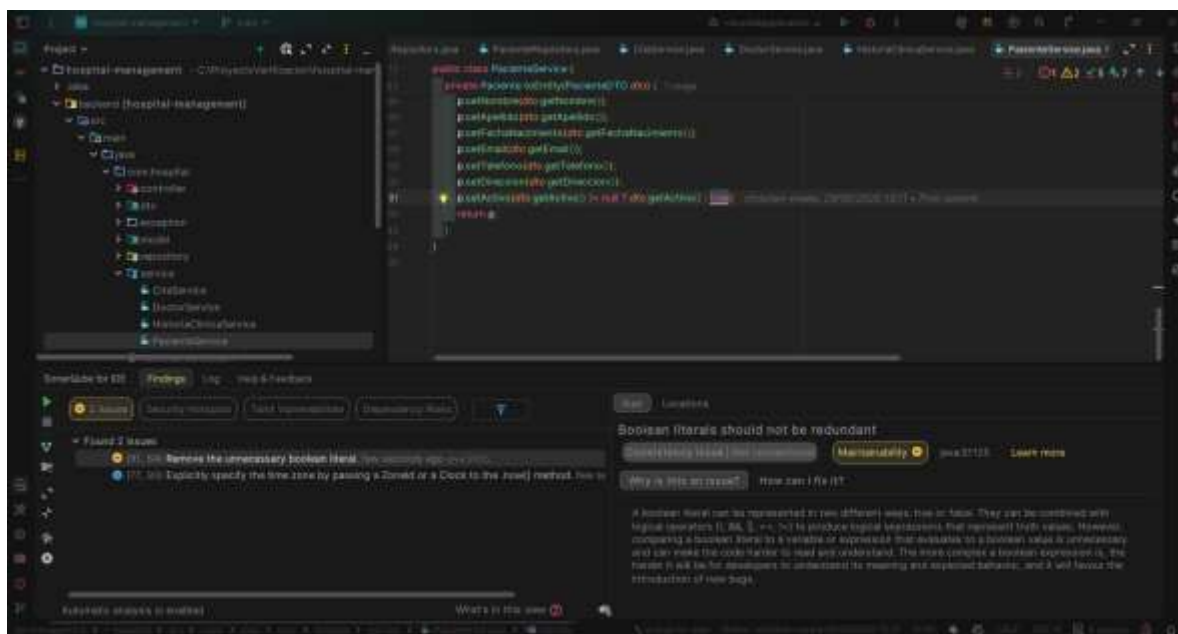


Figura 25. Dos hallazgos registrados en PacienteService.

5.2.6 Clases de src/main sin incidencias

No se encontraron incidencias en ResourceNotFoundException; en los modelos Cita, Doctor y Paciente; en CitaRepository, DoctorRepository, HistoriaClinicaRepository y PacienteRepository; en CitaService e HistoriaClinicaService; ni en HospitalApplication.

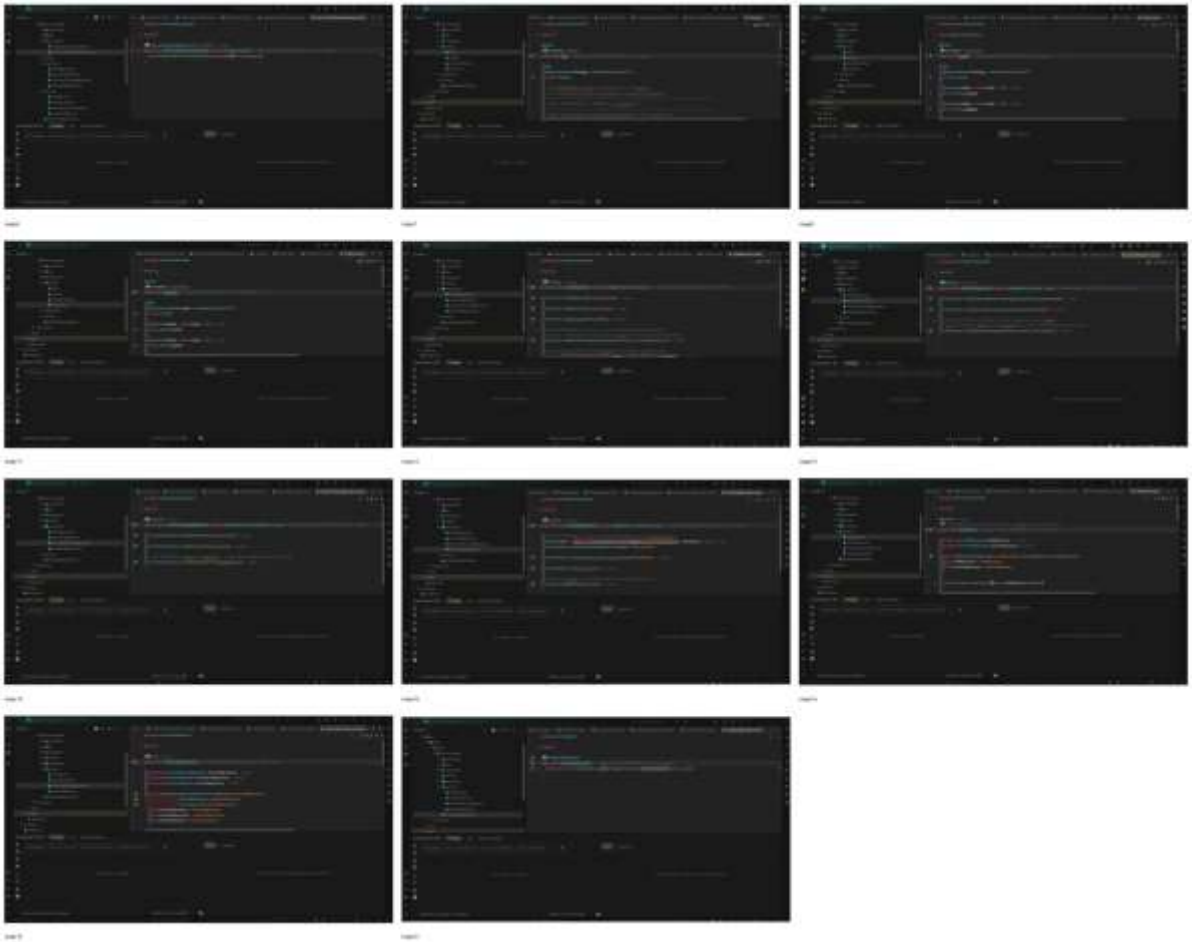


Figura 26. Evidencias de archivos de src/main revisados sin incidencias.

6. Análisis del frontend con ESLint

6.1 Resultado inicial

Se ejecutó el comando `npm run lint` desde la carpeta frontend. El primer resultado presentó 92 errores y 0 advertencias. La mayoría correspondía a las reglas `no-undef` y `no-unused-vars`.

Entre los nombres reportados se encontraban módulos, API y funciones como `PacientesModule`, `DoctoresModule`, `CitasModule`, `HistoriasModule`, `PacientesAPI`, `showAlert`, `formatDate` y `localToISO`. Estas variables se definen en archivos distintos y se comparten mediante etiquetas `script`, por lo que ESLint no las reconocía automáticamente como globales.



```

Terminal Local - Git Bash - Git Bash (2) x +
PS C:\Users\MY LENOVO\Desktop\Programas de Estudio\ProyectoFinalVerificación\hospital-management\frontend> npm run
lifecycle scripts included in hospital-management-frontend:
  test
    jest --runInBand
available via 'npm run-script':
  lint
    eslint .
  lint:fix
    eslint . --fix
PS C:\Users\MY LENOVO\Desktop\Programas de Estudio\ProyectoFinalVerificación\hospital-management\frontend> npm run lint
> lint
> eslint .

F:\Users\MY LENOVO\Desktop\Programas de Estudio\ProyectoFinalVerificación\hospital-management\frontend\src\app.js
60:23  error  'PacientesModule' is not defined  no-undef
61:23  error  'DoctoresModule' is not defined   no-undef
64:23  error  'CitasModule' is not defined      no-undef
67:23  error  'HistoriasModule' is not defined  no-undef
70:17  error  'PacientesAPI' is not defined     no-undef
77:17  error  'DoctoresAPI' is not defined      no-undef
78:17  error  'CitasAPI' is not defined         no-undef
94:48  error  'PacientesAPI' is not defined     no-undef
97:22  error  'e' is defined but never used    no-unused-vars
100:18 error  'error' is defined but never used no-unused-vars

F:\Users\MY LENOVO\Desktop\Programas de Estudio\ProyectoFinalVerificación\hospital-management\frontend\src\citas.js
11:17  error  'CitasModule' is assigned a value but never used  no-unused-vars
27:33  error  'CitasAPI' is not defined                         no-undef
38:18  error  'error' is defined but never used                 no-unused-vars
37:40  error  'DoctoresAPI' is not defined                     no-undef
38:18  error  'e' is defined but never used                     no-unused-vars
43:41  error  'PacientesAPI' is not defined                     no-undef
44:18  error  'e' is defined but never used                     no-unused-vars
46:23  error  'formatDateTime' is not defined                   no-undef
88:33  error  'CitasAPI' is not defined                         no-undef
90:18  error  'e' is defined but never used                     no-unused-vars
91:33  error  'showAlert' is not defined                       no-undef
100:18 error  'localization' is not defined                    no-undef

```

```

Terminal Local * Git Bash * Git Bash (2) * +
130:13 error 'showAlert' is not defined no-undef
139:18 error 'error' is defined but never used no-unused-vars
148:13 error 'showAlert' is not defined no-undef
148:29 error 'HistoriasAPI' is not defined no-undef
152:30 error 'formatDate' is not defined no-undef
174:18 error 'error' is defined but never used no-unused-vars
175:13 error 'showAlert' is not defined no-undef

C:\Users\MY\Idea\workspace\Programas de Estudio\ProyectoFinalVerificacionIdenHospital-management\frontend\js\pacientes.js
11:7 error 'PacientesModule' is assigned a value but never used no-unused-vars
22:37 error 'PacientesAPI' is not defined no-undef
25:18 error 'error' is defined but never used no-unused-vars
33:34 error 'PacientesAPI' is not defined no-undef
36:16 error 'e' is defined but never used no-unused-vars
56:23 error 'formatDate' is not defined no-undef
78:44 error 'PacientesAPI' is not defined no-undef
134:23 error 'PacientesAPI' is not defined no-undef
135:17 error 'showAlert' is not defined no-undef
137:23 error 'PacientesAPI' is not defined no-undef
138:17 error 'showAlert' is not defined no-undef
143:18 error 'error' is defined but never used no-unused-vars
145:13 error 'showAlert' is not defined no-undef
151:34 error 'PacientesAPI' is not defined no-undef
158:34 error 'formatDate' is not defined no-undef
162:18 error 'error' is defined but never used no-unused-vars
163:13 error 'showAlert' is not defined no-undef
169:38 error 'PacientesAPI' is not defined no-undef
171:18 error 'error' is defined but never used no-unused-vars
172:13 error 'showAlert' is not defined no-undef
179:19 error 'PacientesAPI' is not defined no-undef
186:13 error 'showAlert' is not defined no-undef
187:18 error 'error' is defined but never used no-unused-vars
188:13 error 'showAlert' is not defined no-undef

```

✖ 92 problems (92 errors, 0 warnings)



6.2 Configuración aplicada

Para adaptar el análisis a la estructura real del proyecto se creó el archivo `eslint.config.mjs`. Esta configuración no desactiva ESLint; en su lugar, separa el código que se ejecuta en el navegador de los archivos de prueba y declara únicamente los nombres compartidos que realmente existen en el proyecto.

- Activa la configuración recomendada de `@eslint/js`.
- Reconoce las variables del navegador mediante `globals.browser`.
- Declara como solo lectura los módulos, API y funciones compartidas.
- Ignora `node_modules`, `coverage` y `dist`.
- Configura los archivos Jest con `globals.node` y `globals.jest`.
- Mantiene activas reglas de calidad como `no-undef`, `no-unused-vars`, `no-redeclare`, `eqeqeq`, `curly` y `semi`.

6.3 Fragmentos principales de `eslint.config.mjs`

```
import js from '@eslint/js';
import globals from 'globals';

const globalesCompartidas = {
  PacientesModule: 'readonly',
  DoctoresModule: 'readonly',
  CitasModule: 'readonly',
  HistoriasModule: 'readonly',
  PacientesAPI: 'readonly',
  DoctoresAPI: 'readonly',
  CitasAPI: 'readonly',
  HistoriasAPI: 'readonly',
  showAlert: 'readonly',
  formatDate: 'readonly',
  formatDateTime: 'readonly',
  localToISO: 'readonly'
};

export default [
  { ignores: ['node_modules/**', 'coverage/**', 'dist/**'] },
  js.configs.recommended,
  {
    files: ['js/**/*.js'],
    ignores: archivosPrueba,
    languageOptions: {
      ecmaVersion: 'latest',
      sourceType: 'script',
      globals: { ...globals.browser, ...globalesCompartidas, module: 'readonly' }
    },
  },
  {
    files: archivosPrueba,
    languageOptions: {
      ecmaVersion: 'latest',
      sourceType: 'commonjs',
      globals: { ...globals.browser, ...globals.node, ...globals.jest }
    },
  }
]
```



];

6.4 Resultado del frontend

Con la configuración aplicada se eliminaron los falsos positivos relacionados con variables compartidas y con Jest. Las reglas continuaron activas, por lo que ESLint quedó preparado para detectar errores reales en modificaciones posteriores. De acuerdo con la comprobación final realizada, las incidencias presentadas en el frontend fueron solucionadas.

7. Resumen general de resultados

Área	Incidencias detectadas	Acción realizada
Backend - clases de prueba	30	Corregidas y verificadas con “No Issues”
Backend - src/main	18	Documentadas; no se modificó el código del docente
Frontend - ESLint	92	Solucionadas mediante una configuración ajustada al proyecto
Total	140	122 corregidas y 18 reportadas sin modificación

El resultado muestra que la mayor cantidad de problemas del frontend se debía a una configuración incompleta del entorno de ESLint, mientras que en el backend los hallazgos se concentraron en mantenibilidad, manejo de fechas, código innecesario y configuraciones de seguridad.

8. Conclusiones

- El uso de SonarLint permitió corregir 30 incidencias en las pruebas del backend sin necesidad de modificar la funcionalidad principal del sistema.
- Los problemas más repetidos en las pruebas fueron el uso de números para representar meses y la presencia de importaciones, variables o asignaciones innecesarias.
- El análisis de src/main permitió identificar riesgos importantes, principalmente una política CORS abierta, consultas SQL dinámicas, constructores vacíos sin explicación y el uso de fechas sin una fuente de tiempo explícita.
- La configuración inicial de ESLint no representaba correctamente la forma en que el frontend comparte módulos y ejecuta pruebas. Al separar los entornos del navegador y Jest se eliminaron los falsos positivos sin desactivar las reglas de calidad.
- Las evidencias del antes y del después permiten comprobar que las correcciones aplicadas fueron reconocidas por las herramientas de análisis estático.

Anexo A. Reglas principales identificadas

Regla	Descripción resumida	Categoría
-------	----------------------	-----------



Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”

java:S8694	Usar constantes Month en lugar de meses numéricos	Mantenibilidad
java:S1481	Eliminar variables locales sin uso	Mantenibilidad
java:S1854	Eliminar asignaciones innecesarias	Mantenibilidad
java:S2925	Evitar Thread.sleep() en pruebas	Confiabilidad
java:S1128	Eliminar importaciones innecesarias	Mantenibilidad
java:S125	Eliminar código comentado	Mantenibilidad
java:S5786	Usar visibilidad de paquete en pruebas JUnit 5	Mantenibilidad
java:S116	Cumplir la convención de nombres de campos	Mantenibilidad
java:S5122	Restringir CORS a orígenes confiables	Seguridad
java:S1186	Evitar métodos o constructores vacíos sin justificación	Mantenibilidad
java:S8688	Especificar Clock o zona horaria en métodos now()	Confiabilidad
java:S1192	Evitar la duplicación de literales de texto	Mantenibilidad
java:S2077	Evitar consultas SQL construidas dinámicamente	Seguridad
java:S1125	Eliminar literales booleanos redundantes	Mantenibilidad